The Platform Selection Engine

A fully exhaustive, feature-level method for choosing the best enterprise platform for an organization, given its exact situation.

A generalized, research- and math-grounded framework for high-stakes platform and vendor selection. It resolves every capability and every external force into a bounded dollar figure, separates the sticker-price question from the total-cost question, and carries the decision all the way through to an executive readout. Two worked domains are used as running examples: a data-warehouse / ML-platform selection (e.g., Snowflake vs. SageMaker vs. Databricks) and a CRM vendor selection. The method is domain-agnostic.

How to read this / core philosophy

Five rules govern the whole engine:

1. **One shared baseline.** Every vendor is priced off the same organization numbers and the same basket of capabilities. The baseline needn't be perfectly exact; it must be identical across vendors, so if a volume is wrong, every vendor's number moves together and the ranking holds.
2. **Everything resolves to a dollar figure.** No "it depends" survives. Each feature, each external force, becomes a number (or a bounded range).
3. **Three separate verdicts, in sequence:** (a) a features/rates-only winner, (b) an external-cost winner, (c) a combined-cost winner. They can disagree, and that disagreement is the insight (usually: rates are a wash, externals decide it).
4. **Verified vs. estimated, always labeled.** [SOURCED] = confirmed against a live official page or a real organization number. [EST] = modeled/assumed. Anything [EST] with real variability gets lower and upper bounds, not a single fake-precise number.
5. **Bounds discipline (three-point).** For uncertain figures use an optimistic (O), most-likely (M), and pessimistic (P) estimate; carry the O-P band, and where a single number is needed use the PERT expected value $E = (O + 4M + P) / 6$. This is more honest than a lone point estimate and more rigorous than a flat range.

PHASE 0 - Frame the decision (before any vendor is named)

- **0.1 - State the trigger.** Why now? (Incumbent end-of-life/support ending, won't scale, integration breaking, cost, security/compliance gap, merger, new go-to-market motion.) The trigger defines what "better" must mean and becomes the Situation/Complication in the eventual write-up.
- **0.2 - Define success criteria + the decision owner.** What does a win look like in 12 months (adoption %, cycle time, cost/seat, data quality), and who signs? Aim the analysis before opening it.

- **0.3 - Set the evaluation horizon.** Optimize for now, or a 3-5 year roadmap? This changes which capabilities count and over how many months external costs amortize. Pick a number (e.g., 3 years / 36 months) and hold it for every TCO roll-up.
- **0.4 - Set decision constraints.** Hard budget ceiling, mandatory compliance regimes (SOC 2, ISO 27001, HIPAA, GDPR/data residency), deployment model (cloud/on-prem), must-keep integrations. These become knockout criteria.

Where to look: internal, the project charter, the sponsor's stated goals, finance's budget envelope, IT's security/compliance policy, legal on data-residency obligations.

PHASE 1 - Build the organization baseline (the shared yardstick)

The baseline has two vectors: a scale vector (how big) and a run-time / usage-frequency vector (how often each metered thing fires). Both are needed because past core seats, platform cost is typically metered by usage, the same way compute-hours are metered in a data-warehouse model.

1.1 - Scale vector

- Seats by role: primary users, power users, admin/dev, read-only/platform, external portal users
- Records / entities: the core objects the platform manages, at current volume
- Data storage (GB) and file storage (GB)
- Number and type of required integrations
- Geographies / languages / currencies (drives localization + data-residency cost)

1.2 - Run-time / usage-frequency vector (per month, at real volume) This is the baseline volume multiplier in every Phase-4 formula. Capture each:

- Automation/workflow executions (flow runs, triggered actions, approvals)
- API calls (inbound + outbound, per integration)
- Messaging volume (email/SMS/push sends), where applicable
- Report/dashboard refreshes (scheduled + ad-hoc) and any query compute behind them
- AI actions by type: predictive scores generated, draft generations, forecast runs, copilot/agent invocations (usually credit-metered)
- Data-enrichment lookups (enrichment credits)
- Document / e-sign envelopes generated, where applicable
- Telephony minutes, where applicable
- Sandbox count + refresh frequency
- External/community logins per month

1.3 - State the baseline's status + run a sensitivity note. Mark each figure [SOURCED] (from a real system) or [EST]. Then state the sensitivity rule explicitly: because the baseline is shared, moving any volume moves all vendors together, so the ranking is robust to baseline error. Name it a sensitivity analysis; it signals rigor.

Where to look for real numbers: the incumbent system's own admin/usage/billing dashboards (seat counts, storage, API usage, send volume), the data warehouse, finance's current invoices (the truest usage signal), IT's integration inventory, and HR/headcount for seat projections. If a number can't be pulled, estimate it with O/M/P bounds and flag [EST].

PHASE 2 - Enumerate what the platform must do (requirements)

- **2.1 - List every use case as a task/feature, not an abstract category.** Categories are just folders; the unit is the leaf feature.
- **2.2 - Split every feature into two usage tiers.** *Likely-use*, day-one usage (this total decides the near-term comparison). *Future-looking*, roadmap capabilities, priced separately so speculation never inflates the real total.
- **2.3 - Tag each requirement Must-have / Nice-to-have.** Must-haves become knockout criteria; a vendor failing one is eliminated before deep analysis.
- **2.4 - Weight the categories (sum to 100%).** e.g., Integration 25 / TCO 25 / Security-compliance 20 / Usability-adoption 15 / Scalability-roadmap 15. This is the backbone of the Phase-7 scoring matrix. Weights come from the Phase-0 success criteria and the sponsor.

Where to look: interview the actual end users for real workflows; the incumbent's most-used features (usage dashboard) reveal true must-haves; the roadmap owner for future-looking items.

PHASE 3 - Choose which platforms to compare

- **3.1 - The incumbent is contender #0 and the benchmark to beat.** If one exists, every alternative must beat it net of switching cost. If greenfield, define a "do-nothing" baseline instead.
- **3.2 - Pull candidates from trusted external rankings**, use more than one so you're not captive to a single methodology: Gartner Magic Quadrant, Forrester Wave, G2 Grid, Capterra / GetApp / Software Advice (SMB-heavy), TrustRadius / PeerSpot / Nucleus Research (enterprise-leaning, TCO/ROI notes).
- **3.3 - Filter the long list before deep-diving:** organization-size segment (SMB vs. mid vs. enterprise), industry fit, deployment model, and the Phase-2 must-have knockouts. ~15 names to a shortlist.
- **3.4 - Lock the final set:** incumbent + top ~2-3 challengers. Three total is the sweet spot; more makes the Phase-4 engine unmanageable, fewer isn't a real comparison.

PHASE 4 - The per-vendor cost-and-capability engine (the heart)

Run identically for Vendor 1, then verbatim for 2 and 3. The unit of analysis is one individual feature. A category with 40 features generates ~40 priced rows. A full run is hundreds of rows per vendor; that exhaustiveness is the deliverable.

4.0 - Build the per-vendor line-item worksheet. One row per feature: Feature | Category | Native / Add-on / Not-supported / Build | Tier that unlocks it | Billing mechanic | Unit rate [SOURCED/EST] | Baseline qty | Included allotment | Overage rate | Monthly cost (O-P bounds if EST) | Likely vs Future | Quality/caveat note Every row ends in a dollar figure (or a bounded range). "It depends" is resolved using the baseline, not left open.

4.1 - Capability check, per feature. Mark native / add-on / not-supported / needs-a-build, and grade the quality of every "yes" (native one-click vs. native-but-clunky vs. third-party workaround). A "yes" that needs a 3-week build is not the same yes and isn't priced as one.

4.2 - Pricing-mechanic classification, per feature. Resolve for every row which billing type applies:

- Bundled in a seat tier, and which. If it only unlocks at a higher tier, its true cost is the tier-upgrade delta x seats that need it, not zero.
- Paid add-on module, per-seat or flat monthly.
- Consumption-metered, API calls, AI credits, sends, enrichment credits, storage GB (unit rate + usually an included allotment + overage rate).
- Hidden/ancillary, sandboxes, premium support as a % of license, integration-user licenses, storage overages, per-environment charges, implementation-partner fees.

4.3 - The atomic feature-costing formula. For each feature row:

```
feature_monthly_cost =
   tier_upgrade_cost_attributable        (delta to the tier that unlocks it x seats needing it)
 + add_on_fee                            (per-seat x seats, or flat)
 + max(0, baseline_qty - included_allot) x overage_unit_rate     (metered w/ free allotment)
 + baseline_qty x unit_rate              (metered from first unit)
 + ancillary_fees                        (sandboxes, support %, integration users)
```

Illustrative micro-examples (rates placeholder until sourced):

- Predictive scoring (AI credits): 8,000 scores/mo, $0.02/credit, 2,000 included -> (8,000-2,000) x $0.02 = $120/mo.
- Message sends: 200,000/mo, 100,000 included, $0.001 overage -> $100/mo.
- Feature bundled only at Enterprise ($150) vs Pro ($75), 50 seats need it -> ($150-$75) x 50 = $3,750/mo (the "free feature that forces a tier upgrade" trap).
- API: 1.2M calls, 1M included, $0.50/1,000 -> $100/mo.
- Sandboxes: 2 x $500 flat -> $1,000/mo.

- File storage: 800 GB, 100 included, $0.25/GB -> $175/mo. Each is one row. Where a rate is [EST], attach O-P bounds.

4.4 - Explode each category to the leaf and price every feature. No folder gets a single number; every leaf gets its own. This is where the row count runs into the hundreds.

4.5 - Subtotal each category (sum its feature rows).

4.6 - Tag every row Likely vs Future; produce two totals: Likely-use monthly total (decides near-term) and Future-looking monthly total (held separate).

4.7 - Price the full capability ceiling, every native feature, even unused ones. Independently price every feature the platform natively has, at baseline, and sum to the max it could ever bill at this scale (the "ceiling" view). Two payoffs: (1) it surfaces standout differentiators, a vendor whose ceiling includes native AI, an industry cloud, or automation the others lack; (2) it bounds future exposure and shows headroom, without inflating the near-term total.

4.8 - Cost the gaps (not-supported / needs-a-build). For each gap, either estimate a build cost (integration/dev effort, amortized monthly, with O-P bounds) or log a capability risk with severity. A gap on a must-have fires the Phase-2 knockout.

4.9 - Vendor roll-up (each number traceable to rows): Likely-use total / Future-looking total / Full-ceiling total / gap-risk register.

4.10 - Repeat 4.0-4.9 verbatim for Vendors 2 & 3. Same worksheet, formula, baseline quantities. Only tier structure, unit rates, and native/add-on status change.

Where to look, per vendor: official pricing page (published tiers and per-seat rates; note that published pages routinely omit enterprise pricing, add-on module rates, and consumption/overage rates, so a sales quote is needed for the real numbers); official product/feature pages (capability inventory and which tier unlocks each); developer/API docs (real API rate limits, call quotas, sandbox rules, storage allotments/overages); trust/security center (SOC 2 / ISO 27001 / HIPAA / GDPR posture, uptime SLA, subprocessors, data residency); status page (real historical uptime, not the marketing SLA); third-party cost/benchmark and real-world "hidden cost" reports (review sites, analyst TCO notes, community forums for gotchas the pricing page hides). Free trial / developer edition (time-boxed) is the only way to (a) verify capability quality, not just a checkmark; (b) confirm what's actually included vs. marketing; (c) test the real UI/UX for adoption risk; (d) run an integration probe in a sandbox. Treat trial findings as [SOURCED] for capability/quality, still [EST] for full-scale cost.

PHASE 5 - Normalize, compare on rates, and declare the FEATURES-ONLY WINNER

- **5.1 - Force the same basket.** Platforms bundle differently. Price the identical basket of capabilities on each, or seat-price comparisons lie. This is the shared-baseline discipline applied to features.
- **5.2 - Build the comparison table**, Likely-use / Future / Ceiling totals side by side per vendor, each cell carrying its O-P band where [EST].
- **5.3 - Declare the features/rates-only winner** on the Likely-use total (the load-bearing number), with the Future and Ceiling views as secondary context. If the totals land within ~10-15%, say so explicitly: rates are a wash, and the decision will be made by Phase 6.
- **5.4 - Bounds check.** Does the features-only winner still win at the pessimistic end of its band vs. rivals' optimistic ends? If the ranking flips under the bounds, declare it a statistical tie rather than a winner.

PHASE 6 - External / exterior costs, quantified (the part that actually decides it)

Rates are usually a wash; the real spread is in standing up and living with a non-incumbent. Every external force below is converted to dollars over the Phase-0 horizon.

The quantification model: each external cost = a deterministic component + a risk component, both in dollars, both bounded.

- Deterministic = hours x loaded_hourly_rate + one-time fees amortized over the horizon. (Loaded rate = salary x ~1.3 for benefits/overhead / 2,080 hrs.)
- Risk (expected value) = P(event) x cost_if_it_happens, for uncertain items like adoption failure or a forced re-platform.
- Bounds = three-point O/M/P on hours, probabilities, and rates; report the O-P band and the PERT expected value E = (O+4M+P)/6.

Enumerate and quantify each force (the incumbent carries roughly $0 on most of these, that's the incumbency advantage):

- **6.1 - Transition / implementation** (one-time, amortized): configuration, setup, admin build.
- **6.2 - Data migration** (one-time): extract, clean, map, load, validate from the incumbent, often the single biggest hidden cost. Size it off the Phase-1 record counts x a per-1,000-record migration-hours estimate (O/M/P).
- **6.3 - Integration build & maintenance:** one-time build for each required integration + recurring maintenance. Native connector is low; custom middleware is high and forever.
- **6.4 - Skillset gap / staffing:** does the team already have the skill (incumbent = yes), or must you hire / retrain?
- **6.5 - Hiring cost & time-to-hire:** recruiter/agency fee (~15-25% of first-year salary) + the productivity cost of the vacancy during a 2-4 month search.
- **6.6 - Learning-curve / ramp time:** months until the team is productive x the productivity drag during ramp. Incumbent = day-one; challengers = 3-6 months slower.

- **6.7 - User adoption risk (expected loss):** a platform nobody adopts is a total loss regardless of price. P(low adoption) x (license_spent + lost_productivity + re-selection_cost).
- **6.8 - UI / UX quality to productivity:** better UI = faster task completion across every seat, every day. time_saved_per_task x tasks/mo x seats x loaded_rate.
- **6.9 - Security & compliance:** cost to reach required posture + risk-adjusted breach/non-compliance exposure. A vendor lacking a mandated certification may be a hard knockout, not a cost.
- **6.10 - Vendor risk & lock-in / exit cost:** financial viability, roadmap health, EOL risk, price-hike exposure at renewal, and data-portability / exit cost. P(forced switch over horizon) x re-platform_cost.
- **6.11 - Downtime / cutover risk:** revenue/productivity lost during migration cutover and from any uptime gap vs. the incumbent.
- **6.12 - Support quality:** premium support tier cost + the expected cost of slow resolution.
- **6.13 - Opportunity cost / time-to-value:** every month sooner to value has a dollar value. The incumbent's day-one start beats a challenger's 3-6 month ramp.
- **6.14 - Tally external costs per vendor** (deterministic + risk, each with O-P bounds, amortized over the horizon) and declare the external-cost winner. Expect the incumbent to dominate here.

PHASE 7 - Combine, score, validate, and declare the COMBINED WINNER

- **7.1 - Combined multi-year TCO:** (features Likely-use total x horizon months) + all external costs, per vendor, carrying O-P bands. Declare the combined-cost winner. Show all three verdicts together (features-only vs external vs combined); narrate any disagreement, that contrast is the core finding.
- **7.2 - Weighted scoring matrix:** score each vendor 1-5 on each Phase-2.4 weighted criterion (capability fit, integration, security, usability/adoption, scalability, and cost-as-a-score); combine to one number. Keep cost and the scoring matrix, cost alone misses adoption/fit; score alone hides the dollars.
- **7.3 - Sensitivity / bounds test on the decision:** re-run 7.1 at the pessimistic end for the leader and the optimistic end for the runner-up. If the winner survives, it's robust; if it flips, say the top two are a statistical tie.
- **7.4 - Validate before committing:** pilot / proof-of-concept on the top 1-2 with real data and real users ("trust but verify"); reference checks with similar-size / same-industry customers; negotiate tiers, ramp discounts, and, critically, exit / data-portability clauses.
- **7.5 - Conditional decision gates.** Frame the recommendation as gates that (ideally) converge: *Decision 1, scope* (stays core -> incumbent/lowest-switching option; genuinely needs a differentiator -> the platform that has it, only if the roadmap truly requires it); *Decision 2, ownership* (current team owns it -> incumbent; dedicated new talent being hired anyway -> a challenger becomes viable because the skills-cost line is now sunk).

PHASE 8 - Recommend (answer-first)

- **8.1 - Lead with the recommendation and the one-line reason,** then the weighted evidence beneath: "Choose Vendor X. On total cost of ownership, migration, integration, and adoption, not sticker price, it wins."
- **8.2 - Support with three MECE reasons,** each with its number: (1) integration/switching cost, (2) security & compliance fit, (3) TCO + adoption. Attach the combined-cost table and the scoring matrix as backup.
- **8.3 - State the near-term vs. long-term split** (ship on the winner now; revisit if the roadmap/ownership changes) and the conditions that would change the answer.
- **8.4 - Close on the one-line TCO thesis** (recency slot).

Appendix A - Consolidated "where to look" checklist

- **Candidate sourcing / rankings:** Gartner Magic Quadrant, Forrester Wave, G2 Grid, Capterra, TrustRadius, PeerSpot, Nucleus Research.
- **Rates:** each vendor's official /pricing page, then a sales quote for enterprise/add-on/consumption rates the page omits.
- **Capabilities:** official /products & /features pages; developer/API docs for real limits & allotments.
- **Security/compliance & uptime:** vendor trust center + independent status-page history.
- **Hands-on truth:** free trial / developer edition (time-boxed) for capability quality, real inclusions, UI/UX, and a sandbox integration probe.
- **External-cost inputs:** salary data (Glassdoor, Levels.fyi, PayScale, Robert Half) for skills/hiring; implementation-partner rate cards for setup/migration; iPaaS pricing for integrations; finance for revenue-/productivity-per-hour; vendor financials (Crunchbase/filings) for lock-in risk; review sites for adoption/support reality; the compliance officer for mandated regimes.

Appendix B - Discipline reminders

- Label every figure [SOURCED] or [EST]; bound every meaningful [EST] with O/M/P and report the PERT expected value.
- The baseline must be identical across vendors; that's what makes the comparison valid, not its exactness.
- Price to what it would actually cost at the baseline; where variability is real, give lower and upper bounds rather than false precision.
- Three verdicts in order, features-only -> external -> combined, and the story lives in where they disagree.

Addendum - Benchmarked against industry practice: seven gaps closed

After running the engine against published implementation-failure data, RFP/TCO guides, and adoption research, the cost machinery held up (it is more granular than most public methods), but seven things were commonly missing or under-weighted.

A. Elevate adoption & change management to a first-class, heaviest-weighted factor. Roughly 47-50% of enterprise-software implementations fail to deliver value, and only ~6-10% of failures are technical; the rest are adoption, bad data, and process. Traditional projects spend ~80% of effort on configuration and ~20% on adoption, which is backwards. Changes: adoption/usability should carry the largest single weight; promote adoption risk (6.7) and UI/UX productivity (6.8) to top-line factors; add a "complexity tax" penalty (score down any vendor whose power/complexity exceeds the team's needs); add an executive-sponsorship / change-management cost line in Phase 6.

B. New PHASE 6B - Quantify the upside (benefits & ROI), then decide on NET VALUE. The engine priced every cost but never the benefits. A real decision maximizes net value = benefits - TCO, with a payback period, not lowest cost. Anchor stat: licensing is only ~30-40% of actual spend; TCO runs 1.5-2x the initial budget. 6B.1 quantify benefits per vendor (revenue lift, time saved, retention, forecasting accuracy, data-quality gains, with O/M/P bounds); 6B.2 payback & ROI (payback_months = implementation + Year-1 cost / monthly net benefit; full realization typically 13-18 months); 6B.3 decide the combined verdict on net value / payback, not cost alone.

C. Formalize the vendor-evaluation mechanism, RFP + scripted demos. A formal RFP built from the Phase-2 requirements sends the same questions to every finalist for comparable written answers; scripted, scenario-based demos make each vendor run your real use cases so you're not "demo-led"; validate the top 1-2 with references from same-size / same-industry customers.

D. Front-end rigor, stakeholder needs assessment + MoSCoW. A structured cross-functional needs assessment before scoring; upgrade prioritization from Must/Nice to MoSCoW (Must / Should / Could / Won't).

E. Data quality & governance (not just migration). Budget dedupe/cleanse effort before migration, not just transfer; name a data owner post-go-live, data quality degrades immediately if nobody owns it.

F. Build-vs-buy default. Default to "buy" for ~85% of needs; build only for a genuine competitive moat. Custom builds run 40-60% higher TCO over 3 years and carry a higher compliance burden in regulated industries.

G. New PHASE 9 - Post-selection: implementation & ongoing governance. Migration checklist (export -> field mapping -> cleanup -> integration testing -> user training); phased go-live (pilot group -> full rollout), not big-bang; a dedicated admin owner (often the single biggest recurring hidden cost); an adoption plan (enablement, real-time user support, executive reinforcement); a review cadence (re-baseline requirements every 12-18 months; renewal-price-escalation watch).

The Quant Layer - every step that can be a formula, is one

Notation: for vendor v over horizon H months. Every [EST] input carries an optimistic O, most-likely M, pessimistic P three-point estimate.

Q1 - Master cost / value equations

Monthly recurring platform cost R_v = sum of feature_monthly_cost (Phase 4.3, every leaf)
One-time costs Oc_v = stand-up + data migration + integration build
Monthly recurring external cost X_v = admin salary + pipeline maint + premium support + change-mgmt
Total Cost of Ownership TCO_v = Oc_v + sum_{t=1..H} (R_v + X_v)
Monthly benefit B_v = revenue-lift + time-saved*rate + retention gain (Phase 6B)
Net Value NV_v = sum_{t=1..H} B_v - TCO_v
Payback (months) PB_v = (Oc_v + first-year recurring) / monthly_net_benefit

Q2 - Feature cost (restated, Phase 4.3)

feature_monthly_cost = tier_upgrade_delta*seats + add_on_fee
+ max(0, qty - included)*overage_rate + qty*unit_rate + ancillary_fees

Q3 - Three-point (PERT) + risk-adjustment (Phase 6)

Expected value E = (O + 4M + P) / 6
Std. dev (rough) SD = (P - O) / 6 -> report the [O, P] band on every EST line
Risk line = P(event) * cost_if_it_happens (adoption failure, forced re-platform, breach)

Q4 - Discounting for a multi-year horizon (optional rigor)

NPV = sum_{t=0..H} CF_t / (1 + r)^t , monthly r = annual_hurdle_rate / 12

Use the organization's WACC/hurdle rate. Over 3 years the effect is modest, but including it is the correct treatment of future dollars.

Q5 - Deriving the criteria weights rigorously: AHP (not gut feel). Pairwise-compare the criteria on Saaty's 1-9 scale; build the n x n positive reciprocal matrix A. Weight vector w: normalize each column, then average across each row (approximates the principal eigenvector). Consistency check: lambda_max = average over i of (A*w)_i / w_i; CI = (lambda_max - n) / (n - 1); CR = CI / RI, accept if CR <= 0.10, else revise. Random Index (RI) by size n (Saaty 1980): 3 -> 0.58, 4 -> 0.90, 5 -> 1.12, 6 -> 1.24, 7 -> 1.32, 8 -> 1.41, 9 -> 1.45, 10 -> 1.49. A CR <= 0.10 is the mathematically defensible answer to "why these weights?"

Q6 - Scoring the vendors: Weighted Sum Model (WSM). For each criterion i, normalize every vendor's raw value to 0-1: higher-is-better $s = (x - min)/(max - min)$; lower-is-better $s = (max - x)/(max - min)$; qualitative 1-5 $s = score/5$. Composite $V_v = sum_i\ w_i * s_{v,i}$. Highest wins. Report NV_v / TCO_v alongside the composite so the committee sees both the fit score and the raw money. Do not double-count, cost enters the composite once, as its own normalized criterion.

Q7 - Robustness / sensitivity (is the winner real?). Bounds test: recompute V_v at each vendor's O and P ends. If the leader's worst score >= the runner-up's best score, robust win; if the intervals overlap, statistical tie. Weight-swing test: vary each weight +/-10 points; if the ranking holds, the decision is insensitive to weight quibbling.

Q8 - Evidence-based default weights (re-weighted on the failure evidence). Knockouts first (pass/fail, never weighted): must-have features, mandated compliance regimes, hard budget ceiling, required deployment model. Weighted criteria among survivors (sum = 100):

Criterion	Weight	Why this weight
Adoption & usability (incl. change-mgmt)	30	~90% of failures are people/process/adoption, not technology
Total cost of ownership / net value	25	Licensing is only 30-40% of real spend; TCO runs 1.5-2x budget
Integration & data fit	20	Integration lead-times and bad data are top failure causes
Scalability & roadmap fit	12	Real but future-facing; discounted vs. present-day adoption/cost
Vendor viability, support & lock-in	8	Renewal escalation + exit cost matter, but rarely decide it
Security/privacy quality beyond the knockout baseline	5	Most security is handled as a pass/fail knockout

For a real decision, derive organization-specific weights via the AHP procedure in Q5 and show the CR.

PHASE 10 - Communicating the decision to leadership (the delivery layer)

The analysis is worthless if leadership doesn't absorb and trust it. This phase runs on the same answer-first discipline as the rest of the engine.

10.1 - Audience laddering: who gets what depth.

Tier	Who	What they care about	Depth	Format
Board / C-suite	CEO/CFO/CIO, board	Net value, risk, strategic fit, "what do you need from me?"	Answer + top 3 drivers + the ask	1-pager or 5-6 slide exec summary
Steering committee / sponsor	Decision owners	Recommendation + trade-offs + TCO + risk + implementation plan	Moderate	~6-slide deck
Working team / IT / procurement	Implementers	Everything, feature tables, formulas, sources, gaps	Full	The complete engine document

10.2 - The "information at a premium" rule. Load-bearing test: include a fact only if omitting it would change the recommendation or a reasonable person's confidence in it. Everything else goes to appendix/backup. The higher the audience, the higher the bar.

10.3 - The exec deck architecture (6-8 slides, one message each, answer-first, every title states the takeaway): (1) Title + recommendation; (2) Scope, what/who was evaluated; (3) Cost comparison, one chart, one message; (4) The decider, TCO / exterior-cost contrast; (5) Decision + gates + horizon; (6) Ceiling & gaps / risk; (7-8, optional) benefits/ROI & payback, implementation plan & the ask. Per slide: title = takeaway, one chart or contrast, <= ~30 words of body, one accent color, caveats in a footnote.

10.4 - Slide-time & length math. 1 slide <= 60s (30-60s sweet spot); attention wall ~6-7 minutes, so a tight exec readout is ~6-10 slides / ~8-12 min + Q&A. If a slide needs more than a minute to talk across, it's really two slides. Put the recommendation on slide 1 and restate it on the last (primacy + recency).

10.5 - Updating leadership along the way. Cadence: a short written status at each phase gate. Format: BLUF, three lines, where we are / what changed / what we need from you. No-surprises rule: pre-wire the recommendation with the sponsor before the group readout.

10.6 - Prioritization: what to cut (from the bottom up until it fits): never cut the recommendation + the ask; then the one decider (TCO/adoption); the 2-3 supporting reasons, one number each; the top risk + mitigation; scope; method (compress to "full model available on request"); feature-level tables, formulas, sources (always appendix).

10.7 - Storytelling for trust. Structure: SCQA opening to answer-first pyramid. Honesty markers: label [SOURCED] vs [EST]; show bounds; cite sources; state the counter-case and why it loses; say explicitly "here's what would change my recommendation." Restraint: one message per slide; let the chart carry it (one highlight color, label > legend). Calibration earns credibility, the analyst who shows the seams is trusted more than the one who over-claims.

10.8 - Q&A / depth on demand. The full engine document is the backup deck; every headline claim should trace to an appendix line or hyperlink. Anticipate the top ~5 questions and pre-build a backup slide for each.

10.9 - One internal benchmark beats a pile of external studies. Executives trust their own organization's data more than third-party research. Lead the readout with one organization-grounded number, and reserve external benchmarks for corroboration.

10.10 - Evidence base. Answer-first sequencing, tailoring by altitude, brevity, detail-to-backup, structure over polish, accuracy as credibility, and narrative for retention are each independently supported in the research on executive communication. (Note: avoid the widely-repeated "stories are 22x more memorable" figure, it is apocryphal; a real study found roughly 63% recall for stories vs. ~5% for a lone statistic, and a 2024 peer-reviewed study found data storytelling improves comprehension accuracy.)

Quant Layer II - Advanced decision-analytic methods (optional depth)

The base Quant Layer gives point estimates with O/M/P bounds. These methods turn those bounds into probabilities and decision-analytic outputs.

- **QII.1 - Monte Carlo simulation (bounds -> probability of winning).** Feed each O/M/P input as a distribution (triangular or PERT-Beta), draw one random value per input, compute TCO_v and NV_v, repeat N ~ 10,000 times, and output P(vendor v has the best net value). Converts "Vendor A wins by a range that might overlap" into "Vendor A has an 82% probability of being the best choice."
- **QII.2 - Expected Value of (Perfect) Information.** EVPI = E[value | perfect information] - value of the best choice under current uncertainty; EVPPI = the same for perfect information on ONE input. Rule: run a pilot on an input only if the pilot's cost < the EVPPI of that input.
- **QII.3 - Tornado / one-way sensitivity.** Vary each input across its O-P range with others held at M; rank inputs by how far they swing the decision metric. The longest bars are the load-bearing variables.
- **QII.4 - Break-even / indifference thresholds.** Solve for the input value at which two vendors tie ("Vendor B only wins if adoption exceeds 85%"). A threshold is more decisive than a range.
- **QII.5 - Scenario analysis.** Define 2-3 internally consistent scenarios and recompute the winner in each; recommend the option that wins across all (robust) or wins in the most probable one.
- **QII.6 - Real-options framing for the staged path.** "Adopt the low-switching-cost incumbent now, revisit later" has option value: it preserves the cheap right to switch when the roadmap clarifies.
- **QII.7 - Finance metrics executives recognize.** Alongside NPV and payback: ROI% = Net Value / TCO; IRR = the discount rate at which NPV = 0.
- **QII.8 - Alternative weighting & scoring methods.** ROC (rank-order centroid) swing weights as a lighter alternative to AHP; TOPSIS as an alternative to WSM (rank by relative closeness to the ideal and anti-ideal solutions). If WSM and TOPSIS agree, the ranking is robust to method choice. Normalization note: min-max is intuitive but outlier-sensitive; z-score or vector normalization is steadier.

- **QII.9 - Inter-rater reliability.** If several evaluators score independently, measure agreement (variance per criterion, or percent-agreement / kappa). High disagreement flags an under-defined criterion.

Research-grounded method upgrades

- **R1 - Predict adoption with TAM/UTAUT instead of guessing P(adoption).** Ground the adoption probability in the Technology Acceptance Model (Davis, 1989): use is predicted by perceived usefulness + perceived ease of use. During the pilot, measure both for each vendor (short user survey + observed task completion). UTAUT2 (Venkatesh) adds performance expectancy, effort expectancy, social influence, facilitating conditions, habit, and price value.
- **R2 - Structure the adoption plan on a change-management model.** Follow ADKAR (Awareness -> Desire -> Knowledge -> Ability -> Reinforcement) or Kotter's 8 steps rather than ad-hoc effort.
- **R3 - Classify requirements with the Kano model (complement to MoSCoW).** Tag features must-be (expected), performance (more is linearly better), and attractive/delighters (differentiators). Stops over-investing in delighters while missing a must-be.

End of engine. A complete, research- and math-grounded methodology for high-stakes platform selection: framing and costing (Phases 0-9), the communication layer (Phase 10), the quant foundations (Quant Layer I & II), and the evidence base under both. This is an original framework by Graham Friedman.